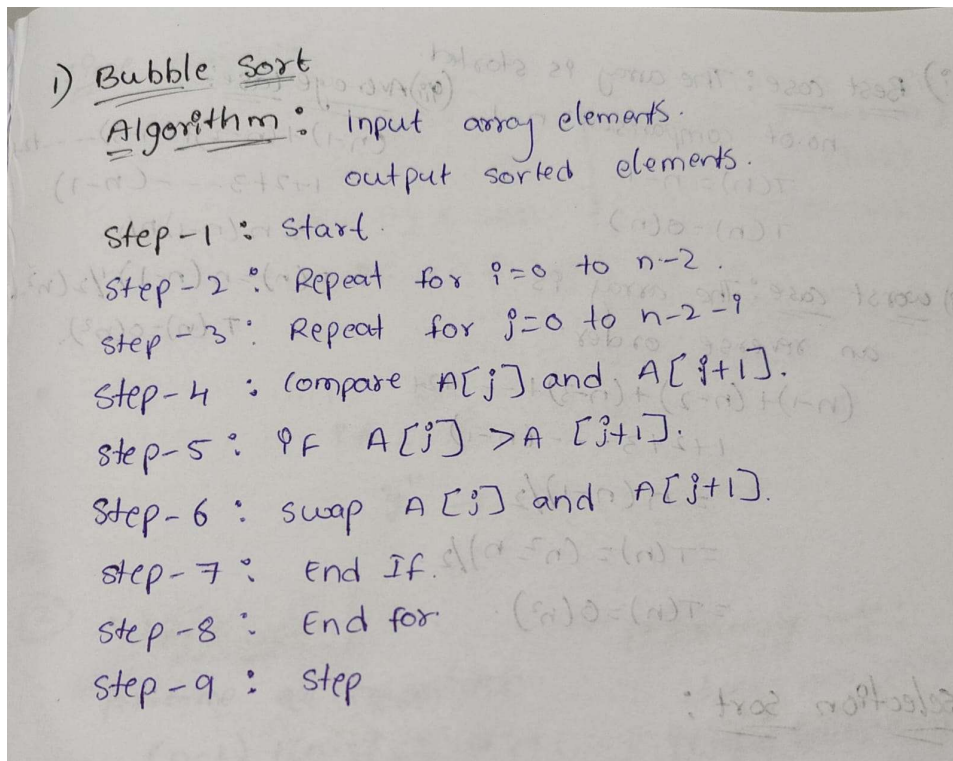


PRACTICAL: 01

Aim: To implement and find the time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quick sort.

A) Bubble Sort:

Algorithm:



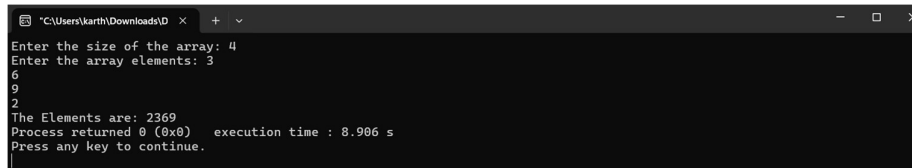
Program / Code:

```
#include<iostream>
using namespace std;
int main()
{
    int a[60], size, i, j, temp;
    cout<<"Enter the size of the array: ";
    cin>>size;
    cout<<"Enter the array elements: ";
```



```
for(i=0; i<size; i++){
    cin>>a[i];
}
for(i=0; i<size-1; i++){
    for(j=0; j<size-i-1; j++){
        if(a[j]>a[j+1]){
            temp = a[j];
            a[j] = a[j+1];
            a[j+1] = temp;
        }
    }
}
cout<<"The Elements are: ";
for(i=0; i<size; i++){
    cout<<a[i];
}
return 0;
}
```

Output:



```
"C:\Users\karth\Downloads\ID" x + v
Enter the size of the array: 4
Enter the array elements: 3
6
9
2
The Elements are: 2369
Process returned 0 (0x0)   execution time : 8.906 s
Press any key to continue.
```

Time Complexity:

Time complexity :

for inner loop
 $i=0; j=0 \text{ to } n-2$
 $\rightarrow n-1 \text{ times}$
 $i=1; j=0 \text{ to } n-2$
 $i=2, j=0 \text{ to } n-3$
 so, $(n-1) + (n-2) + (n-3) + \dots + 1$
 $\Rightarrow (n-1) \frac{(n-1-1)}{2} = T(n)$
 $T(n) = (n^2 - n)/2$
 we ignore constant or take high expo
 $T(n) = n^2$
So, $T(n) = O(n^2)$

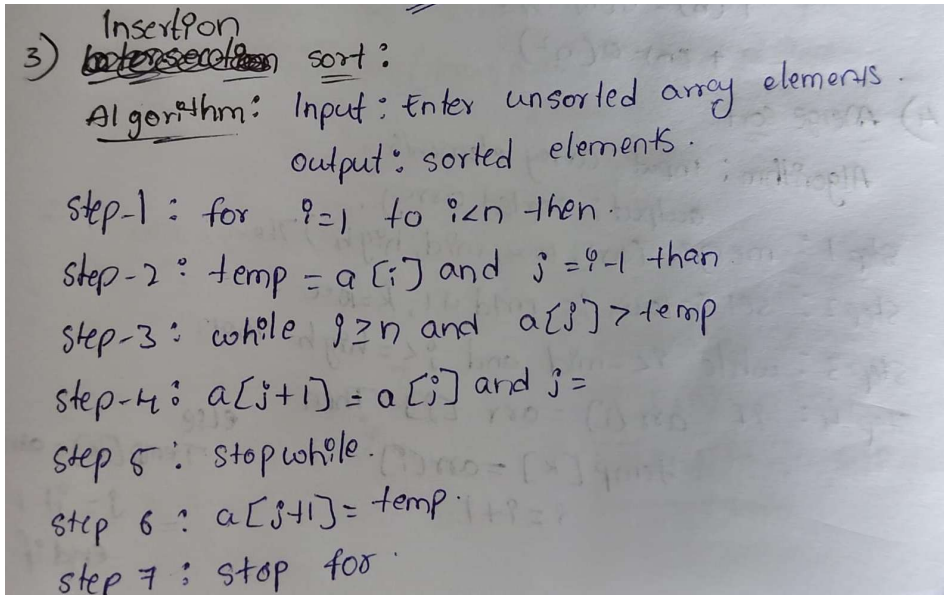
?) Best case: The array is sorted.
 no. of comparison: $n-1$
 $T(n) = n-1$
 $T(n) = O(n)$

(ii) Average case:
 $(n-1) + (n-2) + (n-3) + \dots + 1$
 $1+2+3+\dots+(n-1)$
 $= n(n-1)/2$
 $T(n) = n(n-1)/2$
 $T(n) = O(n^2)$

(iii) worst case: The array is in reverse order.
 $(n-1) + (n-2) + (n-3) + \dots + 1$
 $1+2+3+\dots+(n-1)$
 $= n(n-1)/2$
 $= T(n) = (n^2 - n)/2$
 $= T(n) = O(n^2)$

B) Insertion Sort:

Algorithm:



3) ~~Insertion~~ sort:
Algorithm: Input: Enter unsorted array elements.
output: sorted elements.
step-1: for $i=1$ to $i \leq n$ then.
step-2: $temp = a[i]$ and $j = i-1$ then
step-3: while $j \geq 0$ and $a[j] > temp$
step-4: $a[j+1] = a[j]$ and $j =$
step 5: stop while.
step 6: $a[j+1] = temp$.
step 7: stop for.

Program / Code:

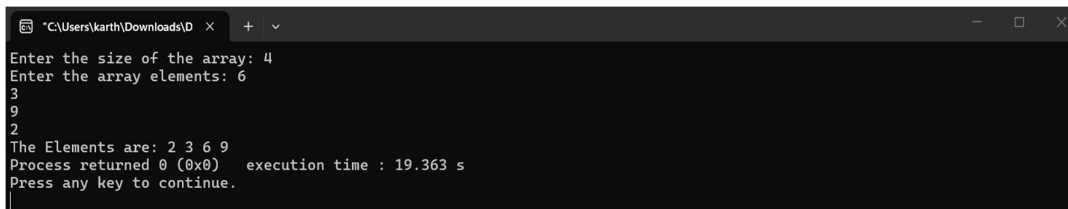
```
#include<iostream>

using namespace std;

int main(){
    int a[60], size, i, j, min, temp;
    cout<<"Enter the size of the array: ";
    cin>>size;
    cout<<"Enter the array elements: ";
    for(i=0; i<size; i++){
        cin>>a[i];
    }
    for(i=0; i<size-1; i++){
        min = i;
```

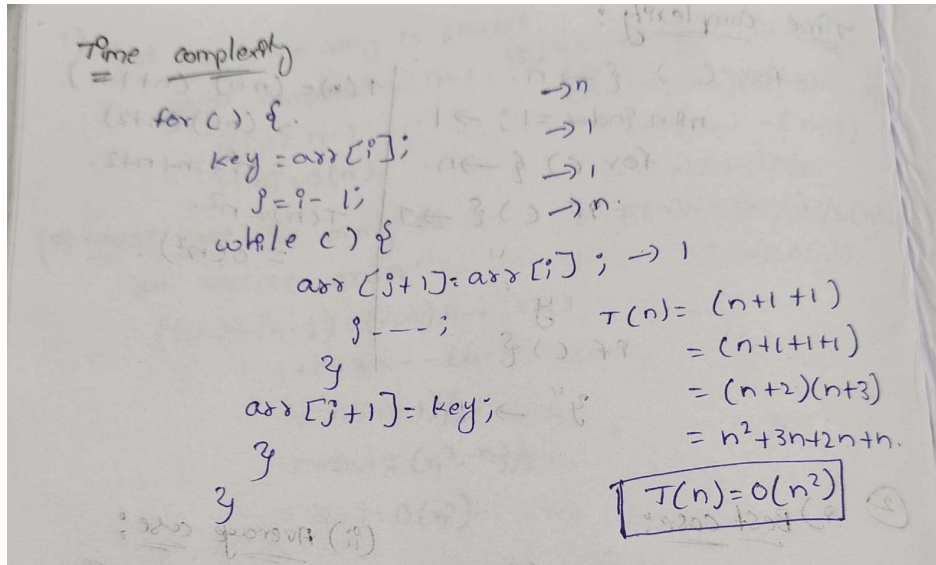
```
        for(j=i+1; j<size; j++){
            if(a[j] < a[min]){
                min = j;
            }
        }
        if(min != i){
            temp = a[i];
            a[i] = a[min];
            a[min] = temp;
        }
    }
    cout<<"The Elements are: ";
    for(i=0; i<size; i++){
        cout<<a[i]<<" ";
    }
    return 0;
}
```

Output:



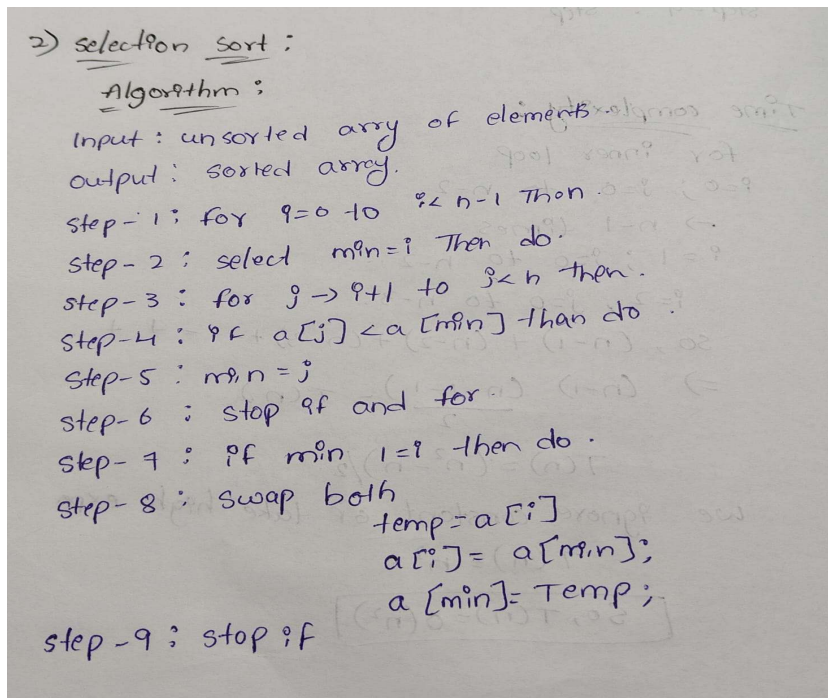
```
Enter the size of the array: 4
Enter the array elements: 6
3
9
2
The Elements are: 2 3 6 9
Process returned 0 (0x0)   execution time : 19.363 s
Press any key to continue.
```

Time Complexity:



C) Selection Sort:

Algorithm:



Program / Code:

```
#include <iostream>

using namespace std;

void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        int temp = arr[minIndex];
        arr[minIndex] = arr[i];
        arr[i] = temp;
    }
}

int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;
    int arr[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    selectionSort(arr, n);
    cout << "Sorted array: ";
```

```
for (int i = 0; i < n; i++){
    cout << arr[i] << " ";
}
return 0;
}
```

Output:

```
*C:\Users\karth\Downloads\D x + v
Enter number of elements: 4
Enter 4 elements: 3
6
2
9
Sorted array: 2 3 6 9
Process returned 0 (0x0)   execution time : 6.650 s
Press any key to continue.
```

Time complexity:

Time complexity:

for () $\rightarrow n$.
min index = 1, $\rightarrow 1$.
for () $\rightarrow n$.
if () $\rightarrow 1$.
 $T(n) = n^2$.
 $= O(n^2)$.

② Best case:
perform all comparison.
 $(n-1) + (n-2) + \dots + 2 + 1$
 $T(n) = n(n-1)/2$
 $= (n^2 - n)/2$
 $T(n) = O(n^2)$.

③ Average case:
check every pass.
 $T(n) = (n-1) + (n-2) + \dots + 1$
 $= n(n-1)/2$
 $T(n) = O(n^2)$.

④ Worst case:
all comparison.
 $T(n) = O(n^2)$.



D) Merge Sort:

Algorithm:

Merge Sort:
Algorithm: Input: array elements.
output: sorted array.
Step 1: merge (arr, low, mid, high) Then.
Step 2: set i=low, j=mid+1, k=low.
Step 3: while i<=mid and j<=high Then.
Step 4: if arr[i] < arr[j] Then
 temp[k] = arr[i]
 i = i+1
else
 temp[k] = arr[j]
 j = j+1
end

k = k+1
end while
Step 5: while j <= high then
 temp[k] = arr[j]; j = j+1; k = k+1
end while
Step 6: while i <= mid then
 temp[k] = arr[i]
 i = i+1
 k = k+1
end while
Step 7: for i=low, k=low
 arr[i] = temp[i]
 i = i+1
Step 8: merge sort (arr, low, high) Then do
Step 9: if low < high then do
 mid = (low + high) / 2
 merge sort (arr, low, mid)
 merge sort (arr, mid+1, high)
end if merge sort (arr, low, mid, high).

Program / Code:

```
#include <iostream>

using namespace std;

void merge(int arr[], int left, int mid, int right) {

    int n1 = mid - left + 1;

    int n2 = right - mid;

    int L[n1], R[n2];

    for (int i = 0; i < n1; i++) {

        L[i] = arr[left + i];

    }

    for (int j = 0; j < n2; j++) {

        R[j] = arr[mid + 1 + j];

    }

    int i = 0;

    int j = 0;

    int k = left;

    while (i < n1 && j < n2) {

        if (L[i] <= R[j]) {

            arr[k++] = L[i++];

        }

    }

}
```

```
        } else {
            arr[k++] = R[j++];
        }
    }
    while (i < n1) {
        arr[k++] = L[i++];
    }
    while (j < n2) {
        arr[k++] = R[j++];
    }
}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;
    int arr[n];
    cout << "Enter elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
}
```

```

    }

    mergeSort(arr, 0, n - 1);

    cout << "Sorted array using Merge Sort: ";

    for (int i = 0; i < n; i++) {

        cout << arr[i] << " ";

    }

    cout << endl;

    return 0;

}

```

Output:

```

C:\Users\karth\Downloads\ID > Enter number of elements: 4
Enter elements: 2
3
6
4
Sorted array using Merge Sort: 2 3 4 6
Process returned 0 (0x0)   execution time : 8.211 s
Press any key to continue.

```

Time Complexity:

Time complexity:

merge sort (low, high):
 if (low < high) {
 mid = low + (high - low) / 2; → (1)
 mergeSort (low, mid);
 mergeSort (mid + 1, high); → 3 log n
 merge (low, mid, high); → n
 }
 while (1) {
 T(n) = (1) (log n) (n)
 T(n) = n log n
 T(n) = O(n log n)

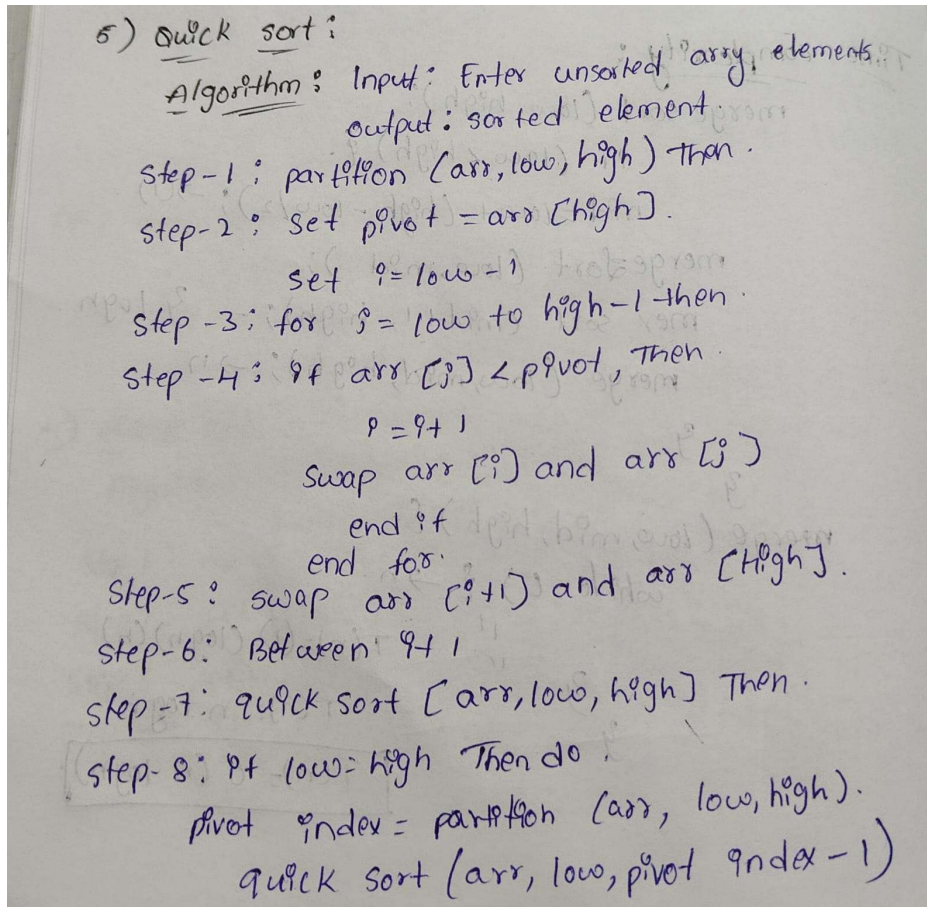
1) Best case: divides array into two halves.
 $n \rightarrow n/2 + n/2 \rightarrow n/4 + n/4 \rightarrow \dots$
 $T(n) = n \log_2 n$
 $T(n) = O(n \log n)$

2) Average case: $n \rightarrow n/2 + n/4 \rightarrow n/8 \rightarrow \dots$
 $T(n) = n \log n$
 $T(n) = O(n \log n)$

3) Worst case: merge sort continuously divides the array into halves. Then merges all elements.
 $n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots$
 $T(n) = n \log_2 n$
 $T(n) = O(n \log n)$

E) Quick Sort:

Algorithm:



5) Quick sort :

Algorithm : Input : Enter unsorted array elements.
output : sorted element.

Step-1 : partition (arr, low, high) Then .

Step-2 : Set pivot = arr [high] .

Set i = low - 1

Step-3 : for i = low to high - 1 then .

Step-4 : If arr [i] < pivot , Then .

 i = i + 1

 Swap arr [i] and arr [high]

 end if

 end for

Step-5 : swap arr [i+1] and arr [high] .

Step-6 : Between i+1

Step-7 : quick sort [arr, low, high] Then .

Step-8 : If low = high Then do .

 pivot index = partition (arr, low, high) .

 quick sort (arr, low, pivot index - 1)

Program / Code:

```
#include <iostream>

using namespace std;

int partition(int arr[], int lb, int ub) {
    int pivot = arr[lb];
    int start = lb;
    int end = ub;
    while (start < end) {
```



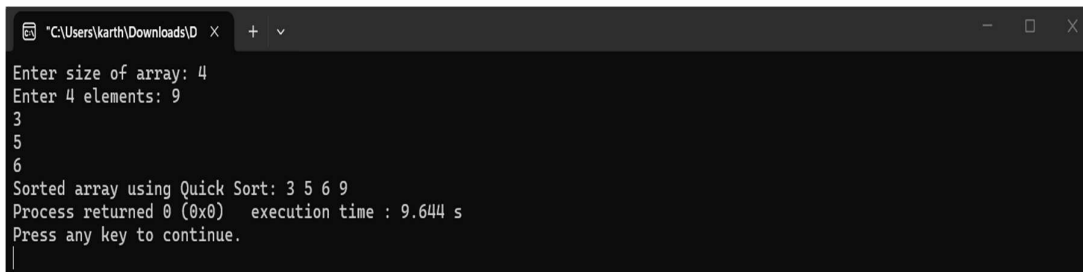
```
        while (arr[start] <= pivot) {
            start++;
        }
        while (arr[end] > pivot) {
            end--;
        }
        if (start < end) {
            swap(arr[start], arr[end]);
        }
    }
    swap(arr[lb], arr[end]);
    return end;
}

void quickSort(int arr[], int lb, int ub) {
    if (lb < ub) {
        int loc = partition(arr, lb, ub);
        quickSort(arr, lb, loc - 1);
        quickSort(arr, loc + 1, ub);
    }
}

int main() {
    int n;
    cout << "Enter size of array: ";
    cin >> n;
    int arr[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) {
```

```
        cin >> arr[i];
    }
    quickSort(arr, 0, n - 1);
    cout << "Sorted array using Quick Sort: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```

Output:



```
"C:\Users\karth\Downloads\D x"
Enter size of array: 4
Enter 4 elements: 9
3
5
6
9
Sorted array using Quick Sort: 3 5 6 9
Process returned 0 (0x0) execution time : 9.644 s
Press any key to continue.
```

Time Complexity:

Time complexity
partition (low, high) {
 //
 for () {
 //
 //
 //
 //
 //
 }
}
quick sort {
 //
 if (low < high) {
 pivot index = partition (low, high);
 quick sort (low, pivot index - 1);
 quick sort (pivot index + 1, high);
 }
}

→ (n-1)
 $T(n) = n \times \log n$
 $T(n) = O(n \log n)$

5) ?) Best case: pivot divides the array into two equal parts.
 $n \rightarrow n/2 + n/2 \rightarrow n/4 + n/4 + n/4 + n/4 \rightarrow \dots + 1$
 $T(n) = n \times \log_2 n$

(ii) Average case: Divides into balance parts.
 $n \rightarrow n/2 + n/2 \rightarrow n/4 + n/4 \dots$
 $T(n) = n \times \log_2 n$

(iii) Worst case: pivot is always smallest / largest element.
 $n + (n-1) \rightarrow (n-2) + (n-3) \rightarrow \dots + 1 + 1$
 $T(n) = (n-1) + (n-2) + \dots + 1 + 1$
 $T(n) = n(n-1)/2$
 $= (n^2 - n)/2$
 $T(n) = O(n^2)$

Conclusion:

In this practical, we studied and implemented five sorting algorithms: Bubble, Selection, and Insertion Sort are simple but slow for large data ($O(n^2)$ time), making them suitable only for small lists. Merge Sort and Quick Sort are much faster for large data since they run in $O(n \log n)$ time. So, simple sorts work fine for small lists, but Merge/Quick Sort are better choices for big data.